Edit PyCR

FullProf PCR Editor

User Manual

A dedicated editor for FullProf `.pcr` refinement files

October 11, 2026

Contents

1 Introduction **3**

2 Getting Started **3**
2.1 Launching the editor . . . 3
2.2 Opening a file . . . 3
2.3 Creating a new file . . . 4
2.4 Saving . . . 4

3 The Main Window **4**
3.1 Docking, floating, and minimizing panels . . . 5
3.2 Tabs and split view . . . 5
3.3 Status bar . . . 6

4 The Menu Bar **6**
4.1 File . . . 6
4.2 Edit . . . 6
4.3 View . . . 6
4.4 Create . . . 6
4.5 PCR . . . 7
4.6 Help . . . 7

5 The Toolbar **8**

6 The Structure Tree **8**
6.1 Basic behaviour . . . 8
6.2 Experiments . . . 9
6.3 Phases . . . 10
6.4 Background . . . 11
6.5 Excluded regions . . . 11
6.6 Number of refined parameters . . . 11
6.7 Constrained parameters . . . 11
6.8 Linear restraints . . . 12
6.9 Box limits . . . 12
6.10 Magnetic and modulated-structure blocks . . . 12

7 Editing Dialogs **12**
7.1 Atoms . . . 12
7.2 Cell parameters . . . 13
7.3 Profile parameters . . . 14
7.4 Instrument parameters . . . 14
7.5 Global and phase commands . . . 14
7.6 Background . . . 15
7.7 Excluded regions . . . 15
7.8 Constrained parameters . . . 15
7.9 Linear restraints and box limits . . . 15
7.10 Pattern weights . . . 15
7.11 Wavelength, radiation and resolution function . . . 15
7.12 Sequential refinement and refinement strategy . . . 15

8 Refinement Codes, Constraints and Renumbering **16**
8.1 What a refinement code is . . . 16

8.2 Constrained parameters, restraints, and box limits . . . 16
8.3 Auto-renumbering . . . 16

9 Running a Refinement **17**
9.1 Run FullProf . . . 17
9.2 Check this file . . . 17
9.3 Edit Results . . . 17
9.4 Quality thresholds . . . 18
9.5 Refinement strategy . . . 18
9.6 Sequential refinement . . . 19
9.7 Refinement summary . . . 20
9.8 Recovering from a stuck run . . . 20

10 Refinement History and the Pattern Panel **20**
10.1 Refinement history . . . 20
10.2 The pattern panel . . . 21

11 Creating New Files **24**
11.1 From a template . . . 24
11.2 From a CIF or magnetic CIF . . . 24
11.3 Combining files into a multi-phase refinement . . . 24
11.4 Combining files into a multi-pattern refinement . . . 24

12 Editor Conveniences **24**
12.1 Syntax highlighting and code colouring . . . 24
12.2 Keyword tooltips . . . 25
12.3 Folding . . . 25
12.4 Show only this . . . 25
12.5 Find and Replace . . . 25
12.6 Column selection mode . . . 25
12.7 Zoom and themes . . . 25

13 A Typical Workflow **25**

14 Keyboard Shortcuts **26**

1 Introduction

Edit PyCR - FullProf PCR Editor (*Edit PyCR* for short) is a dedicated text editor for FullProf `.pcr` refinement files. A `.pcr` file is a plain-text, column-based format that describes everything a Rietveld (or profile-matching, or single-crystal) refinement needs: the experiment, the instrument, the phases, the atoms, the background, and which numbers in the file are free to refine. It is dense, positional, and unforgiving of a misplaced column – exactly the kind of file a general-purpose text editor has no special knowledge of.

This program does not replace FullProf itself. FullProf's own programs (`fp2k` and friends) still do the actual crystallographic calculation. What this editor adds is everything around that calculation:

- A **structure tree** that reads the file and shows its experiments, phases, background, constraints and restraints as a navigable outline, with the right editing action one right-click away from whatever it acts on.
- **Purpose-built dialogs** for the things that are painful to edit as raw columns of numbers – atoms, cell parameters, profile parameters, instrument parameters, background points, refinement commands.
- **Syntax highlighting and code-aware coloring** that distinguishes ordinary values from refinement codes, and keyword tooltips drawn from FullProf's own documentation.
- Tools for **running a refinement** from inside the editor, with a live quality readout, a before/after comparison, and a history of every accepted result.
- Tools for **creating** a new `.pcr` from a CIF, an mCIF, a template, or by combining existing files into a multi-phase or multi-pattern refinement.

How this manual is organised

The manual follows the program's own layout: the main window and its panels first, then the menu bar and toolbar, then the structure tree (the part used most, and where most editing happens), then the dialogs it opens, then running a refinement, and finally the tools for creating new files. It does not explain how FullProf's own refinement algorithm works, or the crystallographic meaning of every keyword – only what each menu, panel and dialog lets the user *do*, and how those pieces fit into a normal working session. A short end-to-end workflow is given in Section 13 for anyone who wants the short version first.

2 Getting Started

2.1 Launching the editor

The editor is a Python application built on PySide6 (Qt). Depending on how it was installed, it is started either with a `fullprof-editor` command, or by running `run.py` from the project folder. No project file or workspace needs to be created first – it opens directly onto an empty window with no tabs, ready to open or create a `.pcr`.

2.2 Opening a file

A `.pcr` can be opened in any of the usual ways:

- **File > Open...** (`Ctrl+O`), or the `Open` toolbar button.
- **File > Open Recent**, which remembers previously opened files.

- **Dragging and dropping** one or more files onto the window – onto an existing tab area, an empty tab strip, or directly onto the structure tree.
- Double-clicking an entry in the **refinement history** panel to load an older, archived state of a file already open (Section 10.1).

Every open file gets its own tab. Several files can be open at once, and the window can be split into two side-by-side tab groups (**View > Move to Other View**) to compare or drag phases between two files at the same time.

2.3 Creating a new file

A `.pcr` rarely starts from nothing. The **Create** menu (Section 11) offers the normal starting points: a plain `New` blank tab, a conversion from a crystal structure file (CIF or magnetic CIF), or a template drawn from a known-good example file. Existing files can also be combined into a multi-phase or multi-pattern refinement.

2.4 Saving

File > Save (`Ctrl+S`) and **File > Save As...** (`Ctrl+Shift+S`) work as in any text editor. Running a refinement also asks to save first, if needed, because FullProf reads the file from disk, not from the editor's own in-memory buffer – so what gets refined is always exactly what has been saved.

3 The Main Window

The window is built from a menu bar, a toolbar, one or two tabbed areas of open `.pcr` editors, and four dockable panels:

Structure tree (left, always visible) – an outline of every open file, described fully in Section 6. This is the main way to navigate and edit a file without reading raw column positions by eye.

Refinement history (hidden until shown from **View**) – charts of quality statistics (R-factors, χ^2, cell parameters, ...) across every *accepted* refinement of the current file. See Section 10.1.

Pattern (appears automatically as soon as there is a curve to plot – the raw data file even before a first refinement, a `.prf` once there is one) – one tab per pattern for a multi-pattern refinement. See Section 10.2.

Refinement monitor (appears floating, on top of the main window, as soon as `Run FullProf` or `Sequential` starts) – the live progress equalizer while a refinement runs, and the before/after comparison once it finishes (Section 9). Being an ordinary dockable panel like the other three, it can be docked into the main window instead of left floating.

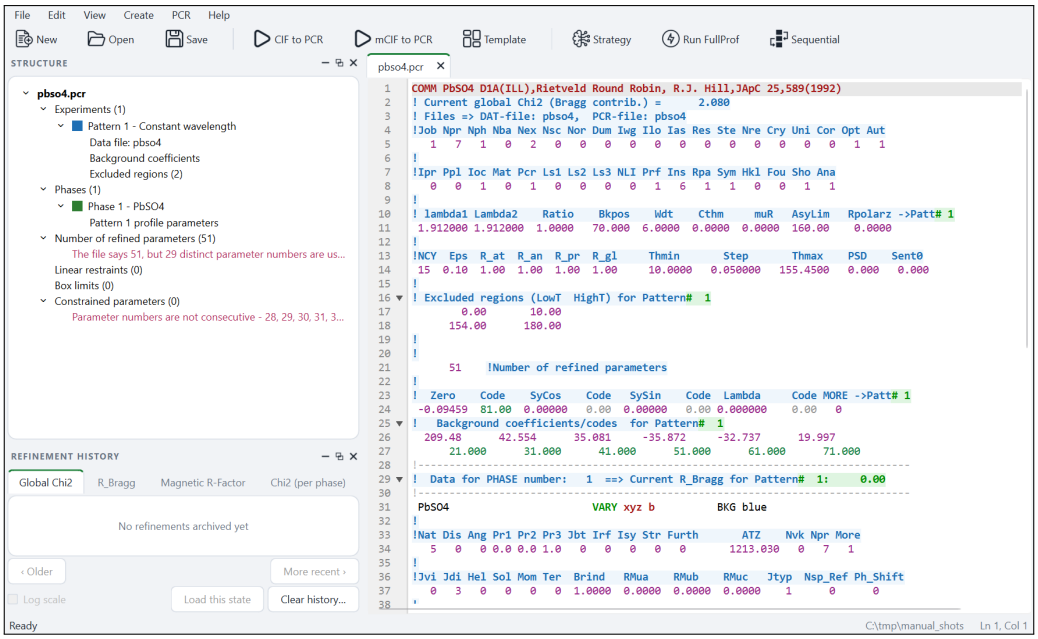


Figure 1: The main window: menu bar and toolbar at the top, the structure tree on the left (a phase and its warnings expanded), the editor in the middle, and the refinement history panel underneath the tree.

3.1 Docking, floating, and minimizing panels

Each panel's title bar has three controls, in the same place a normal window's would be:

- **Minimize** – hides the panel and leaves a small tab at the bottom of the window as the way back. Clicking that tab (or something that needs the panel again, such as a new refinement producing a plot) restores it.
- **Maximize/Restore** – undocks the panel (if it wasn't already floating) and expands it to fill the current screen; clicking it again puts the panel back exactly where it was – docked, or floating at whatever size it had before.
- **Close** – hides the panel entirely; it can only be brought back from the **View** menu, which lists a checkable entry for each of the three panels.

Double-clicking anywhere on a panel's title bar undocks it into its own window (or docks it back in, if it is already floating) – the same as dragging it out or back in, just faster.

A floating panel's own window has a subtle border, so it stays distinguishable from an equally dark (or light) desktop behind it even with nothing else marking where it ends.

Panels can also be dragged by their title bar to a different edge of the window, or dragged out and back in to change their docked size. Dragging one panel's title bar onto another combines them into a single tab group at that spot – useful for keeping, say, Pattern and the refinement monitor together without either one covering the other. Each combined panel keeps its own **View** menu entry, which brings its tab to the front on top of switching it back on.

3.2 Tabs and split view

Every open file is a tab. Tabs can be reordered by dragging, and closed with the usual close button on each tab (or by closing the window, which asks about unsaved changes file by file). Dragging a tab to the very edge of the window – or choosing **View > Move to Other View**

(`Ctrl+Shift+M`) – moves it into a second, independent tab group so two files (or two views of the same layout) can sit side by side.

3.3 Status bar

The status bar at the bottom shows transient messages ("Saved", "Nba is now 22", ...) on the left, and two standing indicators on the right: the cursor's current line/column (and how many lines are selected, if any), and the active file's own folder – useful once several files that happen to share a name, from different refinement attempts, are open at once.

4 The Menu Bar

The menu bar is deliberately short. Anything that belongs to a particular block of a `.pcr` – a phase, a pattern, a background – is reached by right-clicking that block in the structure tree (Section 6), next to the thing it acts on, rather than from a menu that would otherwise grow without limit. The menu bar only carries what applies to the *whole window* or the *whole file*.

4.1 File

- `New` (`Ctrl+N`) – a blank tab.
- `Open...` (`Ctrl+O`) / `Open Recent` – open a `.pcr` from disk, or from the recently-used list.
- `Save` (`Ctrl+S`) / `Save As...` (`Ctrl+Shift+S`).
- `Exit` (`Ctrl+Q`).

4.2 Edit

- `Undo` / `Redo`, `Cut` / `Copy` / `Paste` – standard text editing, all the usual shortcuts.
- `Find...` / `Replace...` / `Find Next` / `Find Previous` – Section 12.
- `Column Selection Mode` – a checkable toggle; while on, an ordinary drag selects a rectangular block of text instead of a normal range (holding `Alt` while dragging always does this, whatever the toggle is set to) – useful for editing a whole column of refinement codes at once.

4.3 View

- Three checkable entries that show or hide the **structure tree**, **refinement history** and **pattern** panels (Section 3).
- `Move to Other View` (`Ctrl+Shift+M`) – moves the current tab into a second, independent tab group.
- `Fold all sections` / `Unfold all` (`Ctrl+Shift+[` / `Ctrl+Shift+]`) – Section 12.
- `Zoom in` / `Zoom out` / `Reset zoom` – also reachable with `Ctrl+scroll`.
- `Show whole file` (`Ctrl+Shift+A`) – undoes a "`Show only this`" action from the structure tree, which temporarily hides everything outside one section (Section 6).
- `Theme` – `Match the system`, `Light` or `Dark`. "Match the system" (the default) follows the desktop's own light/dark setting; the other two override it for this window.

4.4 Create

Starting a *new* refinement from something else is a different activity from editing the file already open, so it has its own menu (and its own toolbar buttons). See Section 11 for what each of these actually produces:

- `From CIF...` – builds a starting `.pcr` from a crystal structure in CIF format.
- `From mCIF (magnetic)...` – builds a starting `.pcr` from a magnetic structure in mCIF format.
- `New PCR from template...` – starts from one of the bundled example files, picked by what it *is* (instrument type, radiation, single-crystal or powder, ...) rather than by filename.
- `Combine into multi-phase...` – merges several `.pcr` files into one refinement with several phases (the same thing dragging a phase from one open file's tree onto another does).
- `Combine into multi-pattern...` – merges several `.pcr` files into one joint, multi-pattern refinement.

4.5 PCR

Actions that apply to the whole `.pcr` file, wherever the structure tree happens to be scrolled to:

- `Check this file` – runs `fp2k` on a throwaway copy of the current file, just to see whether FullProf reaches its own "Normal end" rather than crashing or hanging – the real file and its outputs are never touched (Section 9).
- `Refinement summary` – a read-only window with every refined parameter's final value and uncertainty, background points on their own tab, and the per-pattern Bragg R-factors, all read from the `.out`/`.sum` files FullProf left beside this `.pcr` the last time it was refined (Section 9).
- `Linear restraints...` and `Box limits...` – open the same editors reachable from the structure tree's own "Linear restraints" and "Box limits" nodes (Section 8). These are file-level, not tied to any one phase or pattern, so they are also reachable here regardless of what else is on screen.
- `Strategy` – define which parameters refine at each step of a staged refinement; applies to both `Run FullProf` and `Sequential` (Section 9).
- `Run FullProf` – runs `fp2k` on the current file (Section 9).
- `Sequential refinement...` – refines a numbered series of data files in turn, starting from the current `.pcr` (Section 9).
- `Locate FullProf installation...` – tells the editor where FullProf's own programs (`fp2k`, the CIF/mCIF converters, ...) are installed, if they are not already found on the system `PATH`.

4.6 Help

- `About` – version and licensing information, and a link to open this manual.

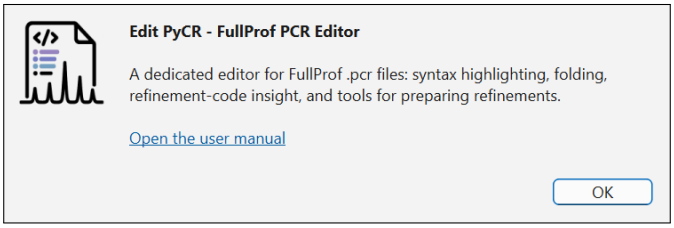


Figure 2: Help > About, with the link to this manual.

5 The Toolbar

The toolbar puts the handful of actions used often enough to deserve being one click away. Most also exist as a menu item described above:

- `New`, `Open`, `Save` – File menu equivalents.
- One button per conversion in the `Create` menu (`CIF to PCR`, `mCIF to PCR`, ...) and `Template` – Section 11.
- `Strategy`, `Run FullProf`, `Sequential` – the three refinement actions, grouped together at the right-hand end of the toolbar (Section 9).

`Run FullProf` is disabled while a refinement is already in progress, to avoid starting a second one on top of it.

Past a second separator, on its own, sits a small icon-only button tinted in warning red: `Kill fp2k` – toolbar-only, with no menu equivalent. It force-closes *every* running `fp2k.exe` process on the machine, not just one this window started, so it asks for confirmation first. This is the escape hatch for a refinement that has hung rather than merely failed – the ordinary `Cancel` button on the progress window only works while that window still holds a live handle to the process, which is exactly what is missing once `fp2k` has crashed into an unresponsive state.

6 The Structure Tree

The structure tree is the main way to work with a `.pcr` without hand-editing raw columns of numbers. It reads the currently open files and shows their experiments, phases and other blocks as an outline, with the editing action that makes sense for each one available through a right-click. It rebuilds itself from the text whenever a file changes, so the tree and the text can never drift apart – there is no separate model of the file to keep in sync.

6.1 Basic behaviour

- **Left-click** a node to jump the editor to the line it represents, switching tabs first if needed. Group headers with no single line of their own ("Linear restraints", "Box limits", ...) simply bring their file's tab to the front.
- **Right-click** a node for the actions available for that kind of block. Most editing in this program starts here.
- A **warning** (shown in red, such as a count that no longer matches what is actually written) always expands the tree enough to be visible, even if the branch it is in was previously collapsed by hand – a problem the tree reports is never left hidden behind a folded node.
- The **active document** (the tab currently being edited) is always expanded; every other open document collapses on its own the moment you switch away from it, so the tree stays focused on whichever file you are actually working on. Expanding a document by hand (clicking its own arrow) while it is *not* the active one keeps it open even after you switch to something else – useful for comparing two files side by side in the tree. Collapsing it by hand again returns it to the automatic behaviour.
- **Dragging** a phase onto another open file's "Phases" group copies it there, turning the two files into a multi-phase refinement without going through the `Combine into multi-phase...` dialog. Dragging an experiment works the same way between files' "Experiments" groups. Phases and experiments can also be dragged to reorder them within the same file. Dragging near the top or bottom edge of the tree scrolls it automatically, so a long list does not have to be pre-scrolled before the drag can reach its target.

- `Show only this` (offered on most nodes) temporarily hides every other line of the file in the editor, so the block can be worked on without the rest of the file in the way. Nothing is removed – it is purely a display choice, undone with **View > Show whole file**.

Every open document is its own top-level entry in the tree; everything below belongs to one of them.

6.2 Experiments

One entry per pattern (`NPATT`) in the file, each showing its instrument type (CW, TOF, single-crystal, ...) and radiation with a small colour swatch. Right-clicking an experiment offers:

- `Set data file...` – which `.dat`/raw data file FullProf should read against this pattern.
- `Edit instrument parameters...` – wavelength(s), zero-point, resolution-function coefficients and the other values that live in the file's Instrumental Parameters section for the profile kind currently in force (Section 7).
- `Load an .irf file...` / `Choose from the .irf library...` – replaces this experiment's instrument parameters with those from a resolution-function file, either browsed for directly or picked from the library bundled with a FullProf installation.
- `Save these parameters to the library...` – the reverse: writes this experiment's current instrument parameters out as an `.irf` file.
- `Change radiation...` – switches between X-ray, neutron CW and neutron TOF radiation types.
- `Convert to single crystal...` / `Convert to TOF...` – changes the fundamental kind of experiment this pattern is.
- `Edit global commands...` – the file-wide `VARY`/`FIX` commands for whole categories of parameters (all background points, all cell parameters, ...), rather than one code at a time.
- `Duplicate this experiment` / `Remove this experiment`.

Each experiment also lists which phases **contribute** to it; right-clicking a phase under an experiment offers `Refine this phase against this pattern`, toggling whether that phase's contribution to this particular pattern is refined at all (relevant for a multi-phase, multi-pattern file where not every phase appears in every pattern).

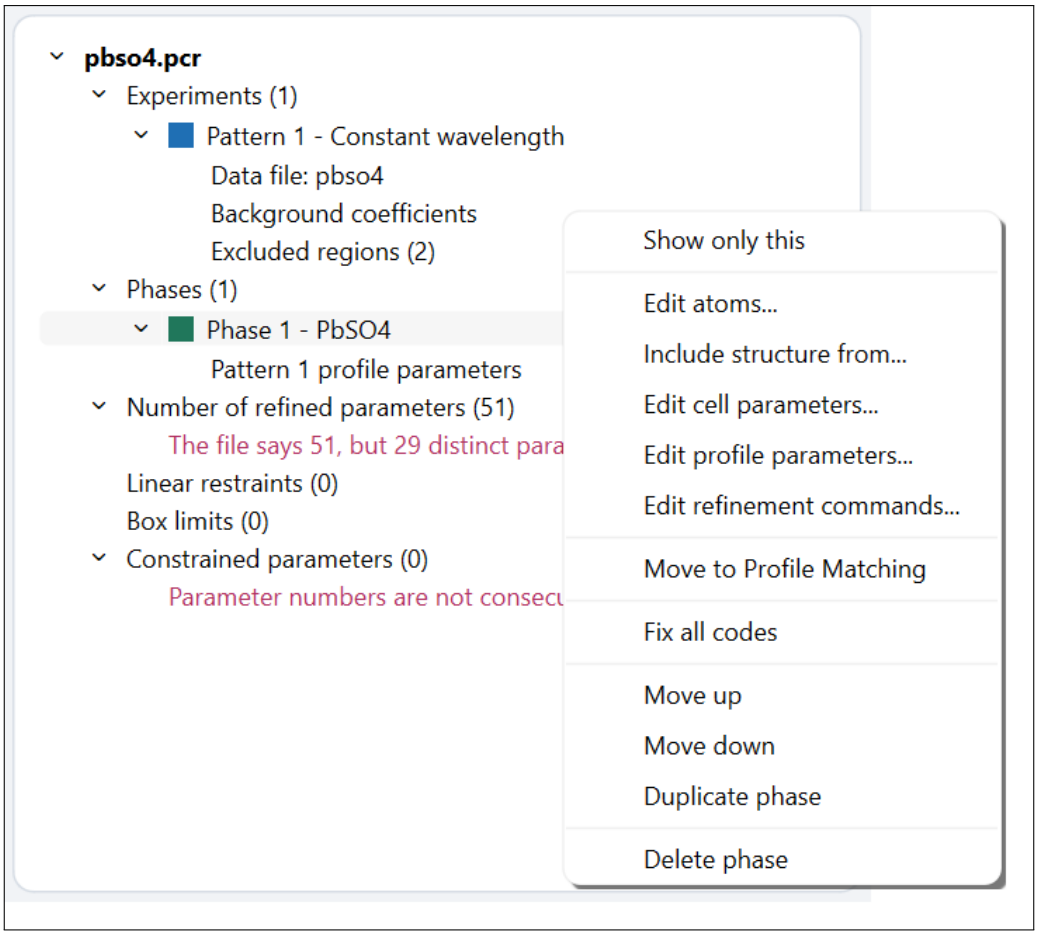


Figure 3: Right-clicking a phase: the menu offers exactly the actions listed below, in the same order.

6.3 Phases

One entry per phase, right-clicking it offers:

- `Edit atoms...` – the atom table (positions, occupancy, displacement parameters, magnetic moments) (Section 7).
- `Include structure from...` – populates an empty phase (0 atoms) from a CIF file.
- `Edit cell parameters...` – $a, b, c, \alpha, \beta, \gamma$, each with its own `Refine` checkbox.
- `Edit profile parameters...` – scale, extinction, overall temperature factor, strain and preferred-orientation parameters for this phase (Section 7).
- `Edit refinement commands...` – the phase-local equivalent of global commands, scoped to this phase's own atoms.
- `Move to Profile Matching` / `Move to Rietveld` – switches the phase's own `Jbt` flag between structure-factor refinement and profile-matching modes (only one of the two is offered, depending on the phase's current mode).
- `Fix all codes` – sets every refinement code belonging to this phase to 0 (fixed), a quick way to stop refining a phase entirely without touching its values.
- `Move up` / `Move down` / `Duplicate phase` / `Delete phase`.

If the atom table's own count of atoms disagrees with the file's declared `Nat`, a red warning appears under the phase with `Update Nat to match`.

6.4 Background

Per pattern, shown either as **Background points** (a table of 2θ/intensity points) or **Background coefficients** (a Fourier/Chebyshev-style set), depending on the background type in use:

- `Import points from a file...` – reads background points out of an external file.
- `Change background type...` – switches between the supported background models.
- `Edit points...` / `Edit coefficients...` – the table editor for whichever kind is active.
- `Refine all points` / `Fix all points` (or `...coefficients`) – sets every background code at once.

A mismatch between the declared `Nba` and how many points are actually written shows the same kind of warning as `Nat`, with `Update Nba to match`.

These text-table editors are not the only way to set a background – the Pattern panel's own `Determine background...` button (Section 10.2) finds a starting set of points automatically, directly on the plotted pattern, which is normally the faster way to get a first background in place.

6.5 Excluded regions

The 2θ/TOF ranges excluded from the fit for a pattern:

- `Add a region...` – prompts for a new excluded range.
- `Edit regions...` – edits the whole list, including removing a region.

The Pattern panel's `Define excluded regions...` button (Section 10.2) reaches the very same block graphically, marking a region directly on the plot instead of typing its limits.

Again, a declared-vs-actual mismatch (`Nex`) shows a warning with `Update Nex to match`.

6.6 Number of refined parameters

A single, file-wide entry (not tied to any phase or pattern) showing the file's own declared count of refined parameters, in the form *Number of refined parameters (N)*:

- If that declared count disagrees with how many distinct parameter numbers are actually used by the codes in the file, a red warning appears with `Update to match`, which corrects the declared count without touching any code.
- Right-clicking the node itself offers `Auto-renumber parameters` – see Section 8.3.

6.7 Constrained parameters

Lists every refinement code number currently shared by more than one place in the file (a genuine constraint, as opposed to an ordinary, independently refined parameter), plus a numbering-problem warning when the numbers in use are not a consecutive run starting from 1.

- Right-clicking the group itself offers `Edit constrained parameters...`, which opens the dedicated editor for creating, changing or breaking a link (Section 7). This node is present whenever the file has *any* refined parameter at all, even before a first link exists, precisely so this is where the first one gets created.

- Right-clicking one linked parameter's own entry offers `Go to the next tied parameter` (steps through every place that shares its number, since they are often far apart in the file) and `Break this link` (gives that one member its own, independent number again).
- A *numbers aren't consecutive* warning offers `Auto-renumber parameters`, the same fix as Section 8.3.

6.8 Linear restraints

The file's `NLI`-declared restraints, each shown as the equation it represents (coefficients and parameter numbers, a target value and its sigma):

- Right-clicking the group offers `Edit linear restraints...` (Section 7).
- Right-clicking one restraint offers `Remove this restraint`.
- A declared/actual mismatch on `NLI` offers `Update NLI to match`.

6.9 Box limits

The file's `Nre`-declared box limits (a parameter clamped to an interval), shown and edited the same way as linear restraints: `Edit box limits...`, `Remove this limit`, and `Update Nre to match` on a count mismatch.

6.10 Magnetic and modulated-structure blocks

For a magnetic or modulated-structure phase, the tree also lists **Polarisation vectors**, **Magnetic basis vectors** and **Mode amplitudes** where the file has them, each navigable the same way as any other section (`Show only this`, click to jump to the block).

7 Editing Dialogs

Every dialog below is reached from the structure tree (Section 6); this section describes what each one is for. Most are non-modal (the window stays open while the rest of the editor is still usable) and write straight through to the file as soon as a value or checkbox changes, rather than collecting a batch of changes to apply on close.

7.1 Atoms

The atom table for one phase: name, type, fractional coordinates, isotropic displacement parameter and occupancy, each with its own `Refine` checkbox, arranged across up to three tabs depending on what the phase's structure-factor mode (`Jbt`) allows:

Position & Occupancy – always present.

Anisotropic ADP – editable per atom, once that atom's own `Anisotropic` checkbox is ticked.

Magnetic moments – present for a magnetic phase; mandatory for a purely magnetic mode, optional (per atom, via a `Magnetic` checkbox) for a combined nuclear+magnetic one.

An atom the editor cannot yet represent safely (an unsupported scattering-factor type, for instance) is still shown, greyed out, with the reason as a tooltip – visible rather than silently dropped.

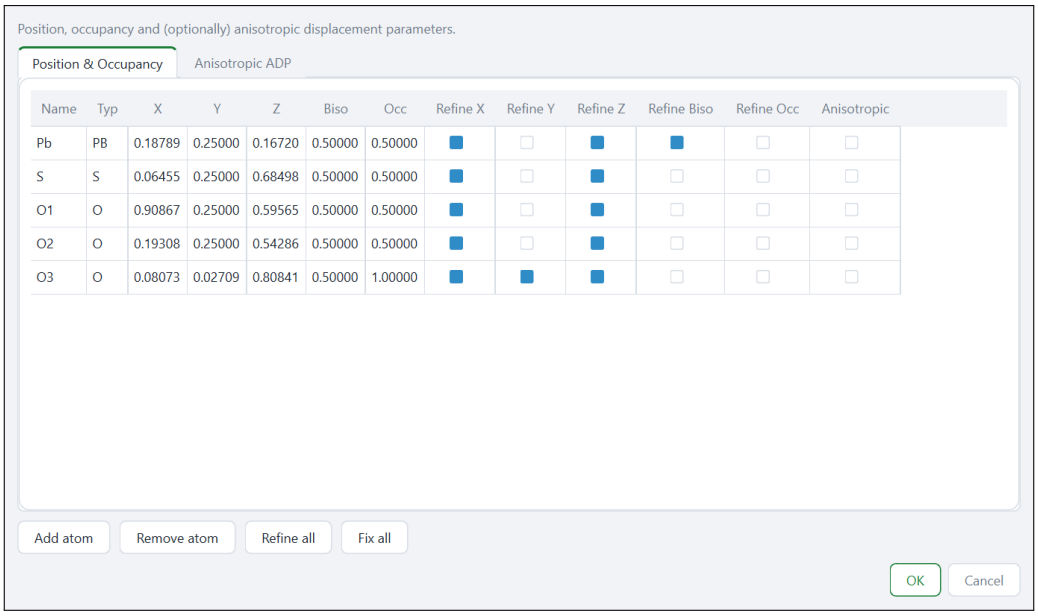


Figure 4: The atoms dialog for an ordinary (non-magnetic) phase – only `Position & Occupancy` and `Anisotropic ADP` are offered; a magnetic phase would also show `Magnetic moments`.

7.2 Cell parameters

$a, b, c, \alpha, \beta, \gamma$ for a phase, each with a `Refine` checkbox. Values are kept as plain text rather than a spin box, so a file's own precision is never silently rounded. If the same phase's cell appears in more than one pattern of a joint refinement and the copies currently disagree, the dialog asks which pattern's values to start from before applying the change to all of them.

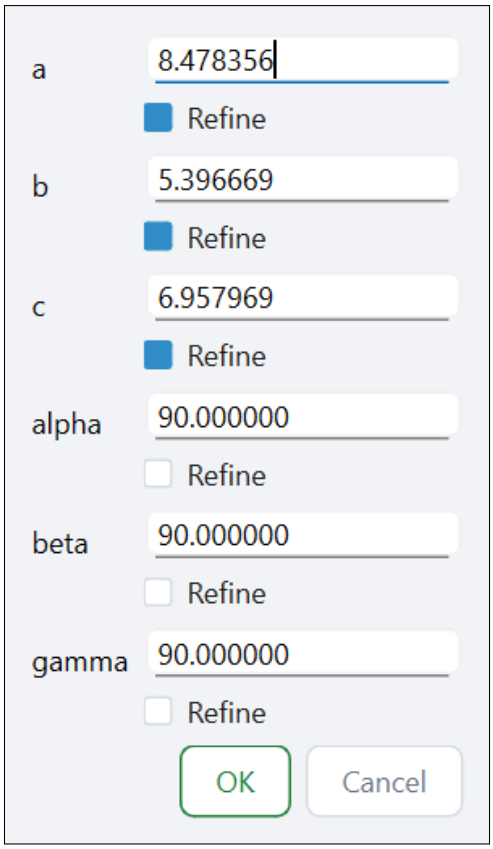


Figure 5: The cell parameters dialog.

7.3 Profile parameters

Scale, extinction, the overall temperature factor, strain (either the three-parameter model or a Laue-class-dependent anisotropic set) and preferred-orientation parameters for one phase, one pattern at a time (a picker lets a multi-pattern phase switch which pattern's own set is being edited). Switching the strain or size model live-updates which fields are shown and enabled to match.

7.4 Instrument parameters

Every field FullProf's Instrumental Parameters section defines for the resolution-function kind currently in force on a pattern – wavelength(s), zero-point and systematic-shift terms, half-width and shape coefficients, and their TOF equivalents. Only VALUES are shown here, not their refinement codes (those are edited from the raw text, or via a global command). A tooltip on each field gives its definition, sourced from FullProf's own documentation.

7.5 Global and phase commands

`VARY/FIX` commands that switch a whole category of parameters on or off at once – every background point, every cell parameter, every atom's isotropic size, and so on – rather than editing one code at a time. The global-commands dialog covers file-wide categories; the phase-commands dialog covers ones scoped to a single phase's own atoms (coordinates, displacement parameters, magnetic moments).

7.6 Background

A table of background points (angle/intensity pairs) or coefficients, depending on the background type currently in use, each row with its own `Refine` checkbox. A separate, small dialog handles switching the background type itself, and another imports a set of points straight from an external file. The Pattern panel's own `Determine background...` button (Section 10.2) reaches a graphical alternative to this table, seeded with an automatic search over the plotted pattern.

7.7 Excluded regions

A table of 2θ (or TOF) ranges to exclude from the fit, with `Add`/`Remove` and direct editing of each range's limits. The Pattern panel's `Define excluded regions...` button (Section 10.2) overlays the same ranges on the plotted pattern instead, for marking one visually rather than typing its limits.

7.8 Constrained parameters

Lists every refinable slot in the file (whether currently refining or not) grouped by where it lives, and lets two or more of them be tied to the same parameter number – with any relative multiplier (for example, one moving as $-1\times$ another) – or have that tie broken again. This is the dialog behind the structure tree's "Constrained parameters" node (Section 6), and the natural place to build a constraint using codes the file already has, rather than typing parameter numbers into the raw text by hand.

7.9 Linear restraints and box limits

Two small, similarly-shaped editors: linear restraints let an equation of the form $c_1 p_1 + c_2 p_2 + \cdots = v(\sigma)$ be written using plain parameter numbers, with a live description of what each number actually is; box limits clamp one parameter to a $[\text{low}, \text{high}]$ interval. Both keep the file's own declared count (`NLI`, `Nre`) in step automatically whenever a restraint or limit is added, edited or removed through the dialog.

7.10 Pattern weights

For a multi-pattern refinement, the relative weight given to each pattern in the joint fit (`W_PAT`).

7.11 Wavelength, radiation and resolution function

- **Change radiation...** switches a pattern between X-ray, neutron CW and neutron TOF, adjusting the fields that only make sense for one or the other.
- **Wavelength** is asked for directly wherever a `.pcr` has no honest existing place to read one from (for instance, right after converting a CIF that carries no wavelength of its own).
- **Choose from the .irf library...** browses the resolution-function files bundled with a located FullProf installation; **Save these parameters to the library...** writes the current experiment's own instrument parameters out as a new one.

7.12 Sequential refinement and refinement strategy

Covered together with the rest of the refinement workflow in Section 9 (the strategy window itself in Section 9.5), since they are steps in that workflow rather than editors of a static block of the file.

8 Refinement Codes, Constraints and Renumbering

8.1 What a refinement code is

Every refinable value in a `.pcr` is followed by a small number, its *code*, that tells FullProf what to do with it:

- **0** – fixed. The value is used as given and never changed by the refinement.
- **A code under 10 in size** (written `1.00`, most commonly) – a placeholder meaning "refine this", used with FullProf's automatic numbering (`Aut=1`): the user marks a parameter this way and FullProf assigns it a real number itself the moment a refinement actually runs.
- **A code of 10 or more** – a real, numbered parameter. The whole part divided by ten is the *parameter number*; the remainder is a *multiplier*. Two codes sharing the same parameter number are refined *together* – their multipliers say how one moves relative to the other (equal multipliers move together, a negative one moves oppositely, and so on).

This editor never asks for a codeword to be typed by hand: every place in this manual that talks about tying two parameters together, or telling a restraint or box limit which parameter it applies to, works in terms of the same plain parameter number, and the dialogs described above take care of writing the actual codeword.

8.2 Constrained parameters, restraints, and box limits

Three different mechanisms build on the same numbering, described in Section 6 and Section 7:

- A **constrained parameter** (a *link*) is nothing more than two or more codewords sharing one number – there is no separate block for it in the file.
- A **linear restraint** is an explicit equation, declared by `NLI`, that ties one or more parameter *numbers* to a target value without necessarily refining them as one.
- A **box limit**, declared by `Nre`, clamps one parameter number to an interval during the refinement.

Because restraints and box limits reference a parameter by its plain *number*, both stay correct automatically as long as that number does not change from under them – which is exactly what the renumbering tool below takes care of.

8.3 Auto-renumbering

Editing a file by hand – adding a parameter here, freeing one there, removing an atom – tends to leave the parameter numbering with gaps, or with numbers far larger than the count of parameters would suggest (a stray `81` in a file that otherwise only uses `1` through `3`, say). `Auto-renumber parameters`, offered from the structure tree's *Number of refined parameters* node and from its numbering-gap warning (Section 6), fixes exactly this:

- Every parameter number currently in use is compacted into a gap-free run starting at 1, in the same relative order they were already in – `1, 3, 5, 81` becomes `1, 2, 3, 4`.
- A **linked group** moves as a whole: every codeword that shared the old number still shares the new one, with its own multiplier untouched.
- Any **linear restraint** or **box limit** that referenced one of the renumbered parameters is updated to the new number automatically, so neither one is left silently pointing at whatever unrelated parameter happens to inherit the old number.
- Any remaining `1.00`-style placeholder is then given its own fresh, independent number, continuing on from the newly compacted range, in the order it appears in the file.

- The file's own declared *Number of refined parameters* is updated to match.

Running it is harmless on a file that is already tidy – it changes nothing when every number already runs consecutively and no placeholder is left to assign.

9 Running a Refinement

A handful of actions, in the `PCR` menu and the right-hand end of the toolbar, cover running FullProf from inside the editor – and looking at, or recovering from, what it did.

9.1 Run FullProf

Runs `fp2k` on the current file. Before launching, the editor:

1. Saves the file if it has unsaved changes – FullProf reads from disk, so what gets refined must match what is on screen.
2. Confirms which data file to refine against, if the `.pcr` does not already name the right one plainly enough to trust.
3. Reads the file's "before" statistics, to compare against afterwards.

While it runs, a live progress panel shows a small grid of quality indicators (a red-to-green "equalizer", one cell per refinement cycle) built from the thresholds described below, so a refinement that is clearly diverging can be seen at a glance rather than only after it finishes. Each cell is a fixed size, so a run with hundreds of cycles (a Simulated Annealing search's own temperature steps, in particular) scrolls horizontally instead of shrinking every cell to nothing – following the latest cycle automatically, unless the strip is scrolled back to look at an earlier one, in which case it stays there until scrolled forward again. In an ordinary refinement the window gives the quality indicators only the height their rows need and the rest to the fit plot below them, so the plot is as large as possible; the handle between the two can be dragged, and once it has been the window stops adjusting the split by itself.

Once `fp2k` finishes, a **before/after comparison** is shown, with two choices:

- **Accept** – loads FullProf's rewritten file into the editor and archives this state in the refinement history (Section 10.1).
- **Discard** – writes the editor's own, pre-refinement version back to disk, undoing whatever FullProf just wrote there, as if the run had never happened.

9.2 Check this file

A lighter, non-destructive alternative to `Run FullProf`: it runs `fp2k` on a throwaway copy of the current file (deleted afterwards, along with everything FullProf wrote beside it) just to see whether the run reaches FullProf's own "Normal end" or fails – useful for confirming a file is set up correctly without touching the real file, its data, or the refinement history. The result is shown in FullProf's own words, in a small transcript window, rather than a second opinion about what might be wrong.

9.3 Edit Results

Edit Results, in the menu bar just before **Help**, opens the files FullProf wrote for the current job – the same as the button of that name in FullProf's own Toolbar. It works like **File > Open...**, but starts in the current file's folder and lists every file named after it (for `job.pcr`:

`job.out`, `job.sum`, `job.prf`, ...), whatever extension FullProf gave it; the chosen files open in editor tabs. With no saved file open, it simply lists every file.

9.4 Quality thresholds

The equalizer's colours come from a small set of user-editable thresholds (per statistic – χ^2, R-factors, and so on – a value below which a cycle counts as "good", and one above which it counts as "bad"). They apply to every refinement run from this editor, single or sequential alike, and are edited from their own settings dialog.

9.5 Refinement strategy

A **strategy** is an ordered list of steps, each one adding more parameters to what `fp2k` refines than the step before it – "first one parameter, then another group, and so on until everything the user wants is refining". Rather than refining everything at once from a possibly poor starting point, a strategy lets a refinement converge on a few parameters first (a scale factor, say) before letting the rest join in.

A strategy applies equally to a single `Run FullProf` (which then runs each step to convergence in turn, automatically) and to a `Sequential refinement...` (Section 9.6 below), so it only needs to be built once. It belongs to the current tab, and reopening `Strategy` shows it again.

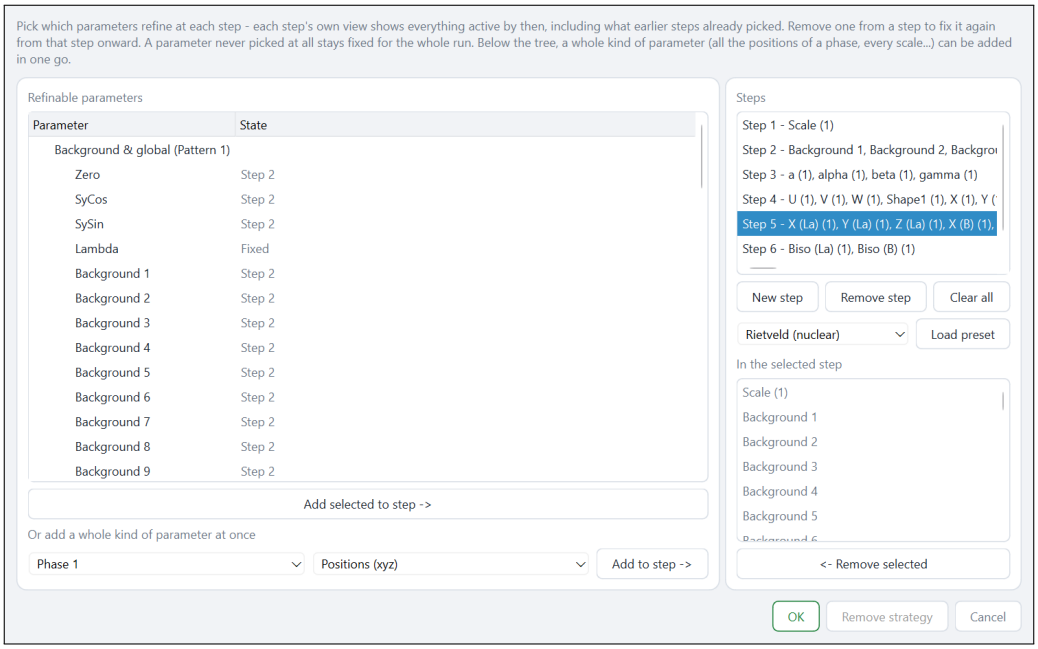


Figure 6: The `Strategy` window with the `Rietveld (nuclear)` preset loaded: the refinable parameters on the left, the steps on the right, and the picker by kind of parameter under the tree.

Building the steps

The left side lists every refinable parameter of the file, grouped phase by phase (so phase 1's `x` is never confused with phase 3's). The right side holds the steps. Select a step, then add parameters to it in either of two ways:

- **One at a time**: select parameters in the tree and press `Add selected to step`, or double-click one.

- **A whole kind at once**: under the tree, choose a *scope* and a *kind* and press `Add to step`. The scope is `Global` (every phase and pattern in the file), a single phase, or, in a file with several patterns, a single pattern. The kinds offered are the ones that exist in that scope: scale, cell, atomic positions, `Biso`/`Uiso`, anisotropic displacement, occupancies, magnetic moments, peak widths, peak shape, asymmetry, preferred orientation, size/strain, overall `B`, extinction, absorption, zero/displacement, background, wavelength, symmetry-mode amplitudes (structural and magnetic) and the propagation vector. So `Global` → `Scale` frees every phase's scale factor, and `Phase 1` → `Positions (xyz)` frees the coordinates of all of phase 1's atoms, without picking them one by one.

Picking a kind simply adds the matching parameters to the step, exactly as if they had been selected in the tree. To leave one out (a light atom whose position should stay fixed, say), select it in `In the selected step` and press `Remove selected`. A parameter FullProf cannot refine for symmetry reasons is switched off by FullProf itself, so asking for it does no harm; parameters the file declares never refinable (a profile-matching scale, for instance) are not offered.

Steps are cumulative: whatever an earlier step added stays active in the later ones, and the view of a step shows everything active by then. Removing a parameter from a step fixes it again from that step onward. A parameter never picked in any step stays fixed for the whole run. In the tree, each active parameter says in which step it joined (`Step 2`); parameters that can never be refined say so and cannot be selected.

Presets

`Load preset` replaces the steps with a typical order of refinement, built from what the file actually contains (a step with nothing to refine in this file is left out):

- `Rietveld (nuclear)`: scale; background and zero; cell; peak widths and shape; positions; `Biso`/`Uiso`; asymmetry and preferred orientation.
- `Le Bail / profile matching`: background and zero; cell; peak widths and shape; asymmetry; size/strain.
- `Magnetic structure`: scale; background and zero; cell; peak widths and shape; magnetic moments; positions and `Biso`/`Uiso`.
- `Symmetry modes`: for a phase described by symmetry-mode amplitudes (`Jbt` $= \pm 6$): scale and zero; cell and overall `B`; the magnetic amplitudes; the displacive amplitudes; peak widths and shape, background and `Biso`. The order of the FullProf symmetry-modes tutorials.

The steps it creates are ordinary steps: edit them afterwards like any others. If there are steps already, the editor asks before replacing them.

Turning the strategy off

`Remove strategy`, next to `OK`, stops refining in stages for this file: it goes back to a single refinement with the codes it has. The button is only enabled when the tab already has a strategy.

9.6 Sequential refinement

Refines a numbered series of data files one after another, each run starting from the previous one's own refined `.pcr` – the same idea as FullProf's own "Sequential" mode (for a temperature or field-dependence series of data sets, typically). The dialog asks for the starting `.pcr`, the naming pattern and numeric range of the data files to work through, and whether a strategy

should apply at every step. A dedicated progress window shows which file is currently refining and lets the whole series be stopped early.

9.7 Refinement summary

`PCR > Refinement summary` opens a read-only window with the full detail behind the last refinement of the current file: every refined parameter's final value and uncertainty (as number(esd), in the same crystallographic notation used throughout the editor), background points on their own tab so they do not drown out everything else, and the per-pattern Bragg R-factors – all read straight from the `.out`/`.sum` files FullProf left beside the `.pcr`. It needs at least one completed refinement to read from, and with several files open it always follows whichever tab is currently active.

9.8 Recovering from a stuck run

The toolbar's `Kill fp2k` button (past a separator, on its own, tinted in warning red) is the escape hatch for a refinement that has genuinely hung rather than merely failed. It force-closes *every* running `fp2k.exe` process on the machine by name, regardless of which window (if any) started it, and asks for confirmation first since it cannot tell one from another. This is different from a progress window's own `Cancel` button, which only works while that window still holds a live handle to the process – exactly what is missing once `fp2k` has crashed into an unresponsive state instead of exiting normally.

10 Refinement History and the Pattern Panel

10.1 Refinement history

Hidden by default – shown from **View > Refinement history** – this panel is for once a refinement's setup is settled and the question becomes "is this run better or worse than the last", rather than something needed while still building the `.pcr`. It draws one chart per quality statistic (χ^2, R-factors, refined cell parameters, ...), one bar per **accepted** refinement of the current file. A declined run never appears here at all – only states that were explicitly kept.

The chart shows at most 15 states at a time – by default the 15 most recent. Once a file has more than that, a thin arrow appears down each side of the bars: it steps the chart one state back (left) or forward (right), keeps going while held down, and is greyed out at the oldest and newest state. The mouse wheel over the chart does the same, one state per notch (up for earlier states). A counter centred below the chart, such as `21-35 / 35`, says which states are showing out of how many. With 15 states or fewer there are no arrows and no counter.

Everything else sits in one compact strip under the chart: `Log` (the log scale, offered on the χ^2 tabs), `Load` (loads the selected bar's state, the same as double-clicking it) and `Clear...` (deletes the whole history, after confirmation); hover over any of them for its full name. When the file's number of phases or patterns has changed over time, the earlier runs are not comparable with the later ones, so they are kept as separate groups: a row with `Older` and `More recent` appears to move between them, and each group is paged the same way.

Double-clicking a bar loads that archived state back into the editor, without touching the file on disk and without losing the tab's own current path: it behaves like any other in-editor edit, exactly as undoable, and exactly as deliberate to make permanent by saving, as anything typed by hand.

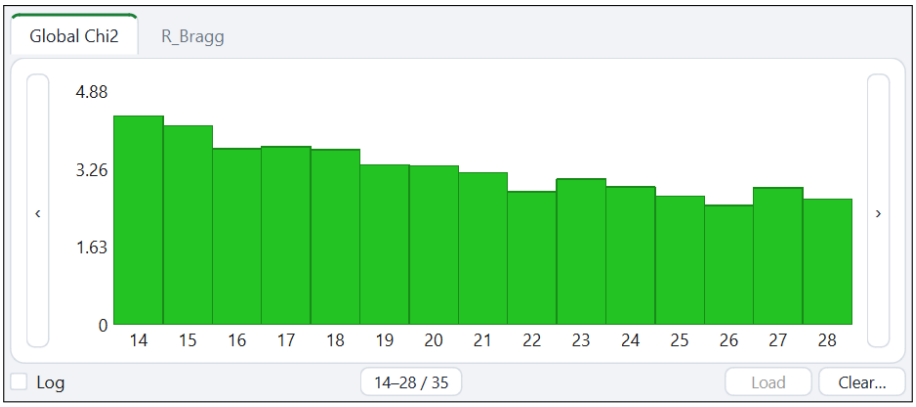


Figure 7: Refinement history for a file with 35 archived refinements, showing states 14–28: the arrows at either side step the window, and the counter below says where it is. Bars are coloured from the same red-to-green quality scale as the live equalizer (Section 9).

10.2 The pattern panel

Shown automatically as soon as there is anything to plot for the current pattern, one tab per pattern for a multi-pattern refinement – and stays out of the way until then. What it plots depends on how far along the pattern is:

- Once FullProf has produced a `.prf` next to the `.pcr`, it plots the observed and calculated curves (and their difference) from that result.
- **Before that** – for a pattern that already names a data file (Section 6's `Set data file...`) but has not been refined yet – it falls back to plotting the **raw data file itself**. This is what makes the panel useful the moment a project's data is loaded, well before a first refinement: there is already a curve on screen to work from.

A pattern whose data file has just been changed and never refined against is treated as "not refined yet" even if an old `.prf` happens to still be sitting on disk, so the plot never shows a stale result left over from a previous data file.

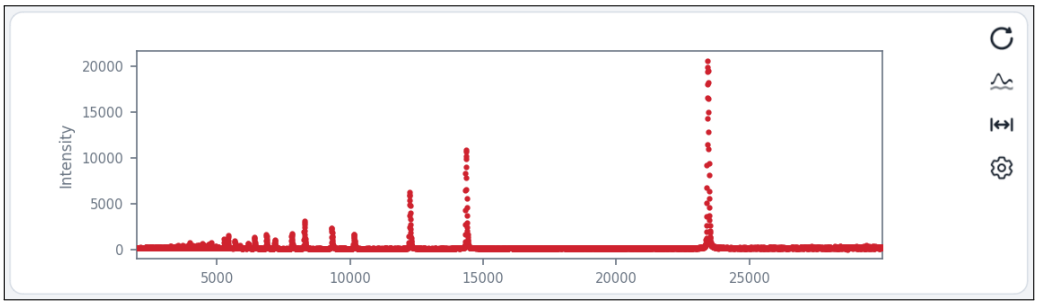


Figure 8: The raw data file for a pattern that has never been refined yet.

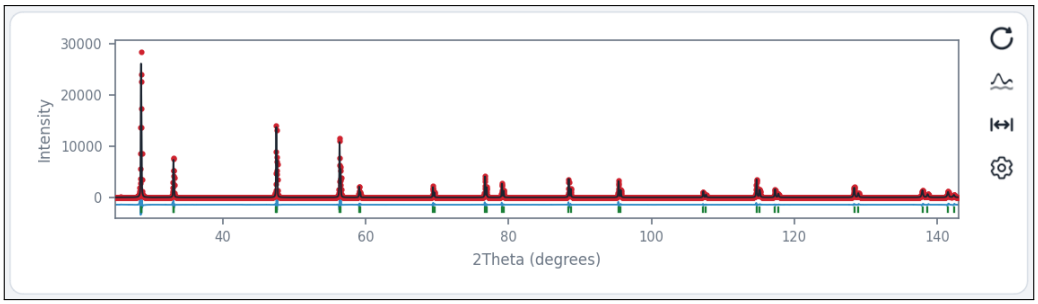


Figure 9: The same panel once a `.prf` exists: observed points, the calculated curve, the difference below it, and reflection positions as tick marks.

Four small icons sit at the edge of the plot:

- `Reset view` – undoes any zooming/panning (right-clicking the plot away from any point or region does the same).
- **Determine background...** – opens a graphical background-points editor for whichever curve is currently shown (raw data or a `.prf`'s own Y_{obs} – either works). It opens already seeded with an **automatic search** for likely background points, which can then be adjusted directly on the plot: left-click adds a point, right-click on a point removes it. Accepting always leaves the pattern with a *points*-type background holding exactly those points, converting it from a coefficients-type background first if that is what it had. This is the fastest way to get a first background in place right after loading a project's data, before ever running a refinement.
- **Define excluded regions...** – opens a graphical editor for the same excluded-regions block the structure tree's own `Edit regions...` reaches (Section 6), started from whatever the file already declares (there is no automatic detection for these – unlike a background, deciding what to exclude is a visual, case-by-case judgement). Double-click marks a region's start, double-click again its end (it is shown shaded once defined); right-click a region to remove it, or right-click away from one to reset the view.
- `Settings` – which curves to draw, log scale, and the other display choices that are not about the data itself.

Both graphical editors also support the zoom every plot in this program shares: scroll to zoom about the cursor, left-drag for a rectangle zoom.

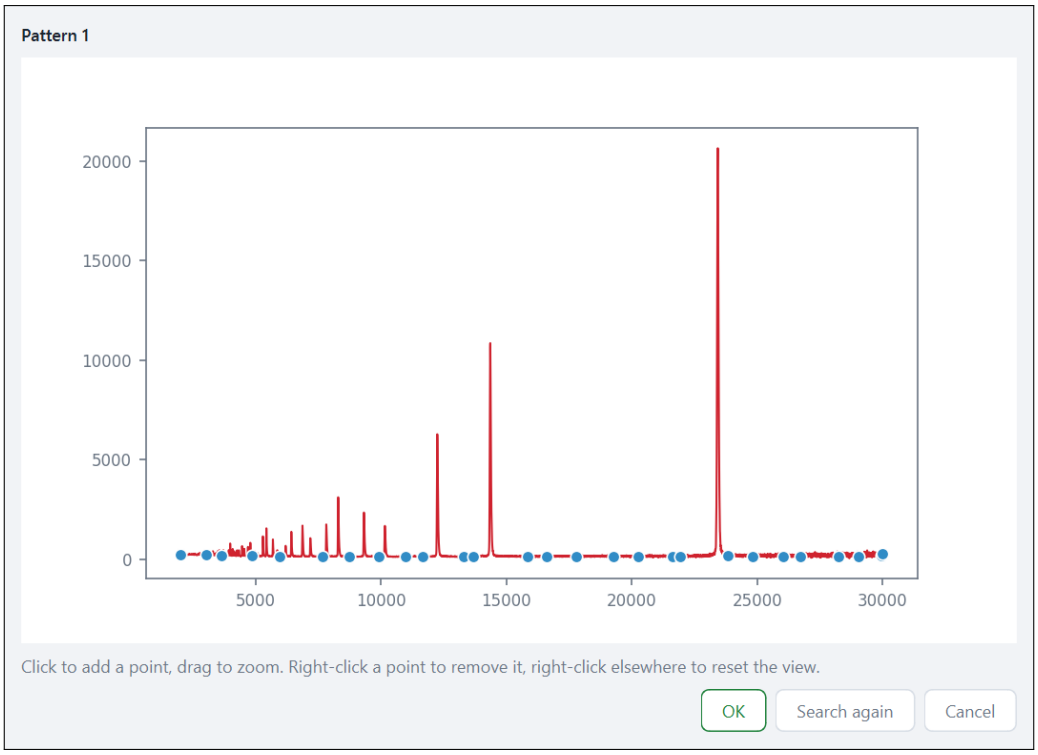


Figure 10: `Determine background...`, already seeded with an automatic search (blue points) over raw TOF data.

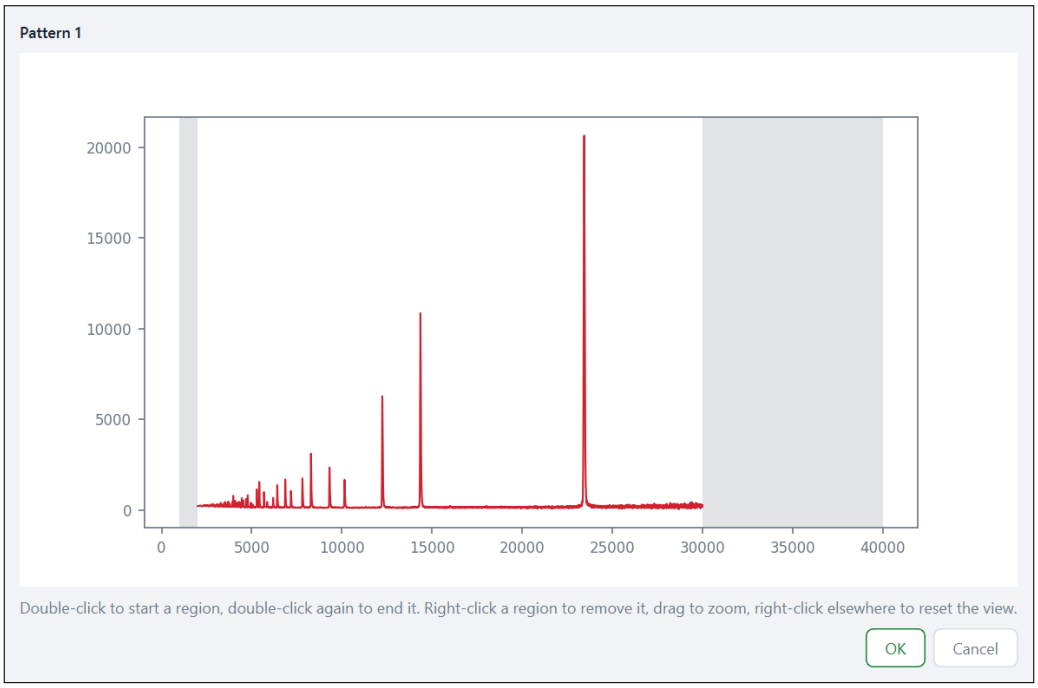


Figure 11: `Define excluded regions...` on the same pattern, showing the two regions this file already declared (shaded).

Loading an older state from the refinement history temporarily shows that state's own plot instead; switching tabs or running a fresh refinement returns the panel to following whatever is currently open.

11 Creating New Files

11.1 From a template

`New PCR from template...` picks a starting `.pcr` from the catalogue of real FullProf example refinements bundled with the program, browsed by *what a file is* rather than by its filename – checkable facets (X-ray/neutron, powder/single crystal, magnetic, number of phases, ...) narrow the list down, and a search box covers the case where a compound's name is what is remembered. Picking one opens it in a new, untitled tab; nothing is written until it is saved for the first time, so the catalogue itself is never modified.

11.2 From a CIF or magnetic CIF

`From CIF...` and `From mCIF (magnetic)...` build a starting `.pcr` from a crystal (or magnetic) structure file, calling FullProf's own conversion programs rather than reimplementing the conversion – so the result can never quietly disagree with what FullProf itself would produce. The conversion window shows the program's own output directly: when a conversion fails, that text is usually the only thing that explains why, and is shown rather than hidden behind a generic error message. Several files can be converted in one pass, each shown as its own row, succeeding or failing independently – the same one-file-per-row layout the live refinement window uses, without the cycle-by-cycle equalizer a plain conversion has no use for.

11.3 Combining files into a multi-phase refinement

`Combine into multi-phase...` picks one already-open file to keep its own instrument settings (the target) and lets a phase be copied in from any other currently open file, one checkbox per candidate phase. This is a second way to do exactly what dragging a phase from one file's structure tree onto another's already does (Section 6) – neither one touches the source file, and the result is always the target file gaining an extra phase, never a brand new document.

11.4 Combining files into a multi-pattern refinement

`Combine into multi-pattern...` lists every currently open, single-pattern `.pcr` with a checkbox; the ones checked (at least two) are folded into one brand new file as a joint, multi-pattern refinement. None of the source files are changed – the result always lands in a fresh tab. A file that is already multi-pattern is listed but disabled, with the reason shown, since combining an already-multi-pattern file into another is a separate piece of work not covered by this first pass.

12 Editor Conveniences

Alongside the structure tree and its dialogs, the raw text editor itself has a few features aimed specifically at `.pcr` files.

12.1 Syntax highlighting and code colouring

Keywords, comments, section headers and phase/atom names are coloured, the same way any programming-language editor highlights its own syntax. On top of that, **refinement codes are coloured differently from the values they belong to** – a code of 0 (fixed), a placeholder (Section 8), and a real, numbered code each read differently at a glance, without needing to compute $\lfloor \text{code}/10 \rfloor$ by eye.

12.2 Keyword tooltips

Hovering a recognised `.pcr` keyword (`Nba`, `Jbt`, `Irf`, ...) in the raw text shows its definition, drawn from FullProf's own documentation. A short definition appears as an ordinary tooltip; a long one opens a small scrollable popup instead, since a plain tooltip cannot be scrolled or have its text selected. The same dictionary also backs tooltips on the equivalent fields inside the instrument-parameters, cell-parameters and profile-parameters dialogs.

12.3 Folding

Every phase and major section can be folded away to a single line, the same as a code editor folds a function body. `Fold all sections` and `Unfold all` (**View** menu, Section 4) act on the whole file at once; the fold margin next to the line numbers folds one section at a time.

12.4 Show only this

Offered from most structure-tree nodes (Section 6): hides every line of the file except the one section chosen, so it can be edited without the rest of a long file scrolling past. This only changes what is displayed – the hidden lines are still there, still saved, still part of undo/redo – and is undone with **View > Show whole file**.

12.5 Find and Replace

Edit > Find... / `Replace...` (Section 4) open the usual search tools, with `Find Next`/`Find Previous` to step through matches once a search term has been entered.

12.6 Column selection mode

While **Edit > Column Selection Mode** is on, an ordinary click and drag selects a rectangular block of text rather than a normal line-based range – useful for editing a whole column of refinement codes at once. Holding `Alt` while dragging does the same thing regardless of whether the mode is toggled on.

12.7 Zoom and themes

`Zoom in`/`Zoom out` (or `Ctrl+scroll`) resize the editor's text; `Reset zoom` returns to the default. The `Theme` submenu switches between following the desktop's own light/dark setting and forcing one or the other for this window – useful in a bright room, on a projector, or for a screenshot that has to match the rest of a document.

13 A Typical Workflow

The pieces above are meant to be used together in roughly this order, though any of them can be revisited at any point:

1. **Start the file.** Either open an existing `.pcr`, or create one from a CIF, a template, or by combining existing files (Section 11).
2. **Point it at the data.** Right-click the experiment in the structure tree and `Set data file...` if it does not already name the right one (Section 6). The pattern panel picks up the raw curve immediately, even though nothing has been refined yet (Section 10.2).
3. **Get a first background and excluded regions in place.** With the raw data already on screen, this is the natural moment to use the pattern panel's own `Determine background...` (finds a starting set of points automatically) and `Define excluded`

`regions...` buttons (Section 10.2) – both work directly on the plot, before a first refinement has even run.

4. **Set up the instrument and phase(s).** Use the structure tree's right-click actions and the dialogs in Section 7 to check or edit instrument parameters, cell parameters and atoms – this is also where a resolution function (`.irf`) gets loaded, and where a phase's atoms get filled in from a CIF if they were not already part of the starting file.
5. **Decide what refines, and in what order.** Turn on refinement for the parameters that should move first (often just a scale factor and the background), leaving the rest fixed. For anything more elaborate than "turn everything on at once", build a `Strategy` (Section 9) instead of managing codes by hand step by step.
6. **Tie parameters together, restrain or limit them, if needed.** Once at least one parameter is refining, "Constrained parameters" in the tree is where two or more get linked together; "Linear restraints" and "Box limits" (also reachable from the `PCR` menu) cover the other two ways to constrain a refinement (Section 8).
7. **Keep the numbering tidy.** After several rounds of freeing and fixing parameters by hand, `Auto-renumber parameters` (Section 8.3) closes any gaps and assigns real numbers to any `1.00` placeholders, before they reach FullProf.
8. **Run it.** `Run FullProf`, watch the live quality equalizer, and **Accept** or **Discard** the result once it finishes (Section 9). An accepted run is archived automatically.
9. **Compare and iterate.** Open the refinement history panel (Section 10.1) to see whether this run improved on the last one; open the pattern panel to see the fit itself. Free more parameters, adjust the model, and run again – reloading an earlier archived state at any point if a change turns out to be a step backwards.
10. **Repeat across a series, if needed.** Once a working model exists for one data set, `Sequential refinement...` runs the same starting point across a numbered series of data files, each one starting from the previous one's own result.

14 Keyboard Shortcuts

Actions marked "platform standard" use whatever shortcut the operating system normally assigns to that action (for example, `Ctrl+S` on Windows/Linux and `Cmd+S` on macOS) rather than a fixed key combination.

Action	Shortcut
New	platform standard (`Ctrl+N`)
Open...	platform standard (`Ctrl+O`)
Save	platform standard (`Ctrl+S`)
Save As...	platform standard (`Ctrl+Shift+S`)
Exit	platform standard (`Ctrl+Q`)
Undo	platform standard (`Ctrl+Z`)
Redo	platform standard
Cut	platform standard (`Ctrl+X`)
Copy	platform standard (`Ctrl+C`)
Paste	platform standard (`Ctrl+V`)
Find...	platform standard (`Ctrl+F`)
Replace...	platform standard (`Ctrl+H`)
Find Next	platform standard (`F3`)
Find Previous	platform standard (`Shift+F3`)
Move to Other View	`Ctrl+Shift+M`

Action	**Shortcut**
Fold all sections	`Ctrl+Shift+[`
Unfold all	`Ctrl+Shift+]`
Zoom in	platform standard, or `Ctrl+scroll`
Zoom out	platform standard, or `Ctrl+scroll`
Reset zoom	`Ctrl+0`
Show whole file	`Ctrl+Shift+A`